

程設一 HW10 task2

C64111174 王宇桐

Function 4 segment(...)

```
void segment(Prefix *grouped_head[33], Prefix *hash_table[33][256]){
    // 初始化 hash_table
    for (int len = 0; len <= 32; len++){
        for (int j = 0; j < 256; j++){
            hash_table[len][j] = NULL;
        }
    }

    // 迭代前面分組完的grouped_head
    for (int len = 0; len <= 32; len++){
        Prefix *current = grouped_head[len];
        while (current != NULL){
            unsigned msb_8_bits = current->ip >> 24;

            // 創建新節點
            Prefix *new_node = (Prefix *)malloc(sizeof(Prefix));
            new_node->ip = current->ip;
            new_node->len = current->len;
            new_node->next = NULL;

            // 插入到對應的 hash_table[len][msb_8_bits]
            if (hash_table[len][msb_8_bits] == NULL){
                hash_table[len][msb_8_bits] = new_node;
            }
            else{
                Prefix *temp = hash_table[len][msb_8_bits];
                while (temp->next != NULL){
                    temp = temp->next;
                }
                temp->next = new_node;
            }

            current = current->next;
        }
    }
}
```

用前面group_len()分組結果，用prefix length當作index，分別將相同prefix length的ip用linked list 串起，變成hash table

1. 初始化hash table
2. 迭代每個prefix length的分組
3. 將該組的data一個一個建立struct prefix後接到hash table

Function 5 prefix_insert(...)

```
void prefix_insert(Prefix *hash_table[33][256], char *s) {
    FILE *file, *insertfile;

    insertfile = fopen("clock_insert.csv", "w+");
    fprintf(insertfile, "clock cycles\n");
    unsigned long long int begin, end;

    file = fopen(s, "r");

    int j;
    unsigned ip1, ip2, ip3, ip4;
    unsigned char len;
    int debugCount = 1;

    while (debugCount <= 4455) {
        begin = read_cntvct();

        j = fscanf(file, "%u.%u.%u.%u/%hhu\n", &ip1, &ip2, &ip3, &ip4, &len);
        if (j != 5) {
            continue;
        }

        // 轉成32位元的IP
        unsigned ip = (ip1 << 24) | (ip2 << 16) | (ip3 << 8) | ip4;

        // hash table index
        unsigned msb_8_bits = ip >> 24;

        // 新節點
        Prefix *new_node = (Prefix *)malloc(sizeof(Prefix));
        new_node->ip = ip;
        new_node->len = len;
        new_node->next = NULL;

        // 對應的hash table
        Prefix **bucket_head = &hash_table[len][msb_8_bits];
```

輸入hash table和要開啟的file，讀取file中每行的ip和prefix length，依照他的prefix length插到對應的hash table位置

1. 讀每行的4個ip和prefix length
2. 將4個ip轉成32位元的一個ip
3. 提取ip的高8位決定hash table index
4. 幫該data建立新節點
5. **bucket_head則是指向要插入的位置

Function 5 prefix_insert(...)

```
// 按 IP 升序排列
if (*bucket_head == NULL || ip < (*bucket_head)->ip){
    // 插入到linked list head
    new_node->next = *bucket_head;
    *bucket_head = new_node;
}
else{
    // 找到插入點
    Prefix *current = *bucket_head;
    while (current->next != NULL && ip > current->next->ip){
        current = current->next;
    }
    new_node->next = current->next;
    current->next = new_node;
}

end = read_cntvct();
fprintf(insertfile, "%llu\n", (end-begin)*(CPU_FREQ/TIMER_FREQ));
debugCount++;
}

fclose(file);
fclose(insertfile);
}
```

因為hash table存的ip要按照大小排序，所以hash table index後要找插入位置

1. 如果小於head, 直接插入在head前
2. 迭代整個linked list找插入點

Function 6 prefix_delete(...)

```
void prefix_delete(Prefix *hash_table[33][256], char *s) {
    FILE *file, *deletefile;

    deletefile = fopen("clock_delete.csv", "w+");
    fprintf(deletefile, "clock cycles\n");
    unsigned long long int begin, end;

    file = fopen(s, "r");

    int j;
    unsigned ip1, ip2, ip3, ip4;
    unsigned char len;
    int debugCount = 1;

    while (debugCount <= 4163) {

        begin = read_cntvct();

        j = fscanf(file, "%u.%u.%u.%u/%hu", &ip1, &ip2, &ip3, &ip4, &len);
        if (j != 5) {
            continue;
        }

        // 轉成32位元的IP
        unsigned ip = (ip1 << 24) | (ip2 << 16) | (ip3 << 8) | ip4;

        // hash table index
        unsigned msb_8_bits = ip >> 24;

        // 對應的hash table
        Prefix **bucket_head = &hash_table[len][msb_8_bits];

        // 初始化當前和前一個節點指針
        Prefix *current = *bucket_head;
        Prefix *prev = NULL;
    }
}
```

輸入hash table和要開啟的file，讀取file中每行的ip和prefix length，依照他的prefix length和ip找到hash table中要刪除的data

1. 讀每行的4個ip和prefix length
2. 將4個ip轉成32位元的一個ip
3. 提取ip的高8位決定hash table index
4. 幫該data建立新節點
5. current和prev分別用來找要刪除的node和前一個node

Function 6 prefix_delete(...)

```
// 持續迭代linked list找到要刪除的節點
while (current != NULL && (current->ip != ip || current->len != len)) {
    prev = current;
    current = current->next;
}

// 如果找到節點
if (current != NULL) {
    // 如果是head node
    if (prev == NULL) {
        *bucket_head = current->next;
    }
    else {
        prev->next = current->next;
    }
    free(current);
}

end = read_cntvct();
fprintf(deletefile, "%llu\n", (end-begin)*(CPU_FREQ/TIMER_FREQ));
debugCount++;
}

fclose(file);
fclose(deletefile);
}
```

1. 持續迭代linked list找到該ip的位置
2. 如果要刪除的node是head的話，直接將head指向下一個node
3. current代表要刪除的node，將prev接到current的下一個node
4. 釋放刪除node的memory

Function 7 search(...)

```
void search(Prefix *hash_table[33][256], char *s) {
    FILE *file, *searchfile;

    searchfile = fopen("clock_search.csv", "w+");
    fprintf(searchfile, "clock cycles\n");
    unsigned long long int begin, end;

    file = fopen(s, "r");

    int j;
    unsigned ip1, ip2, ip3, ip4;
    int debugCount = 1;

    while (debugCount <= 89088) {

        begin = read_cntvct();

        j = fscanf(file, "%u.%u.%u.%u", &ip1, &ip2, &ip3, &ip4);
        if (j != 4) {
            continue;
        }

        // 轉成32位元的IP
        unsigned ip = (ip1 << 24) | (ip2 << 16) | (ip3 << 8) | ip4;
        unsigned msb_8_bits = ip >> 24;

        bool match_found = false;
```

輸入hash table和要開啟的file, 讀取file中每行的ip, 依照他的ip查看是否在hash table中

1. 讀每行的4個ip
2. 將4個ip轉成32位元的一個ip
3. 提取ip的高8位決定hash table index
4. match_found代表有無找到

Function 7 search(...)

```
// prefix length由大到小進行match
for (int len = 32; len >= 0; len--) {
    Prefix *current = hash_table[len][msb_8_bits];
    while (current != NULL) {
        unsigned mask = (len == 0) ? 0 : ~((1 << (32 - len)) - 1);
        if ((ip & mask) == (current->ip & mask)) {
            match_found = true;
            break;
        }
        current = current->next;
    }
    if (match_found) {
        break;
    }
}

// result
if (match_found) {
    found[debugCount] = true;
} else {
    found[debugCount] = false;
}

end = read_cntvct();
fprintf(searchfile, "%llu\n", (end-begin)*(CPU_FREQ/TIMER_FREQ));
debugCount++;
}

fclose(file);
fclose(searchfile);
}
```

從最長的prefix length開始找是否match, 將該length的所有ip逐一比對

1. mask用來確保只比對prefix length是否match, 例如說24的話, 就要用mask將ip後8位設為0
2. 當ip和當前current的ip都等於mask時, 代表match
3. 更新儲存每次結果(0/1)的found(array)

Clock Cycles

因為使用的是arm不是x86, 不能用rdtsc, 所以使用arm上的read_cntvct()

```
#define CPU_FREQ 3200000000ULL
#define TIMER_FREQ 240000000ULL
uint64_t read_cntvct(){
    uint64_t cntvct;
    asm volatile("mrs %0, cntvct_el0" : "=r"(cntvct));
    return cntvct;
}
```

但read_cntvct不是直接從CPU讀取值, 有自己的固定頻率, 所以要再轉換成rdtsc的clock cycles

$$\text{rdtsc_cycles} = \text{cntvct_cycles} \times \left(\frac{\text{CPU 主頻}}{\text{參考頻率}} \right)$$

我的電腦的CPU是3.2GHz, cntvct的相關頻率是2.4MHz

所以read_cntvct()的值再乘上 (3.2G/2.4M)約133.33

Data visualization

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv('clock_segment.csv')
df = pd.DataFrame(data)

stats = df['clock cycles'].describe()

print(stats)

# 分組區間
bins = list(range(200, 7000, 400))

# 將數據分組並計算頻率
df['bins'] = (parameter) figsize: tuple[float, float] | None e) # 分組
frequency = frequency : (float, float), default: :rc: figure.figure
bin_labels = bin_labels : (float, float), default: :rc: figure.figure
Width, height in inches. or interval in frequency.index] # 區間標籤

plt.figure(figsize=(12, 6))
plt.plot(range(len(frequency)), frequency.values, marker='o', color='blue', linestyle='--')

plt.xticks(range(len(frequency)), bin_labels, rotation=45, fontsize=10)

plt.title('Function 4 segment(...)', fontsize=16)
plt.xlabel('Clock Cycles', fontsize=12)
plt.ylabel('Frequency', fontsize=12)

plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Function已經將執行的Clock cycles存在csv

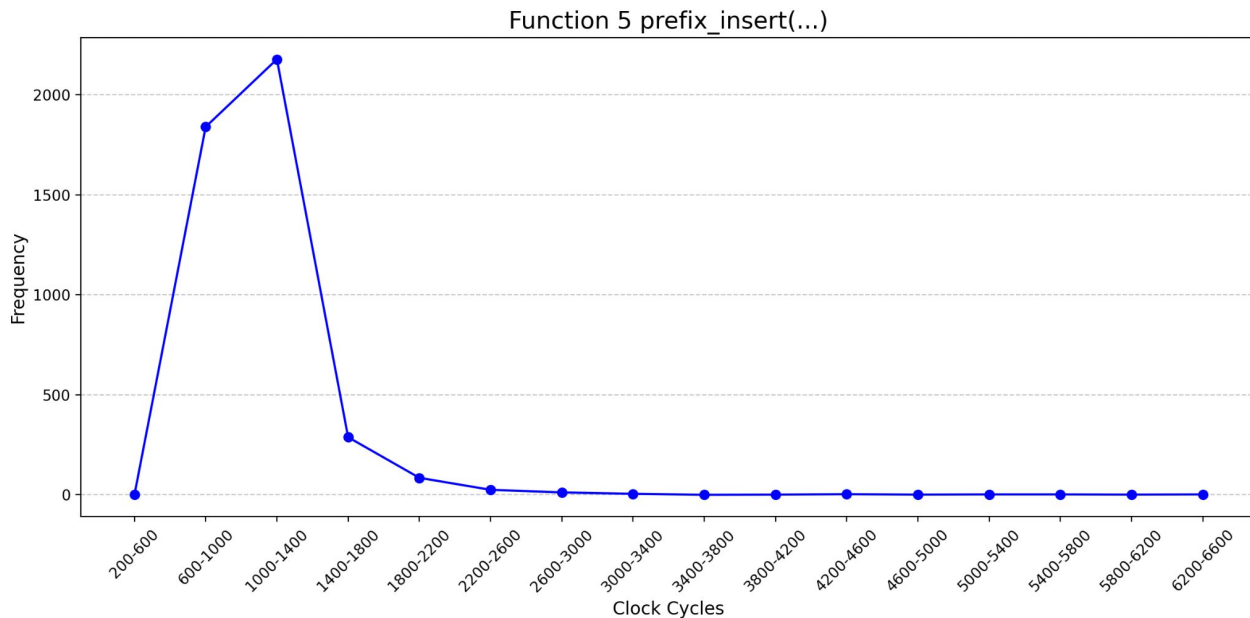
用python讀取csv, 依照range 200-7000每400一組

頻率代表該clock cycles區間出現的次數

以折線圖呈現

Function 5 prefix_insert(...)

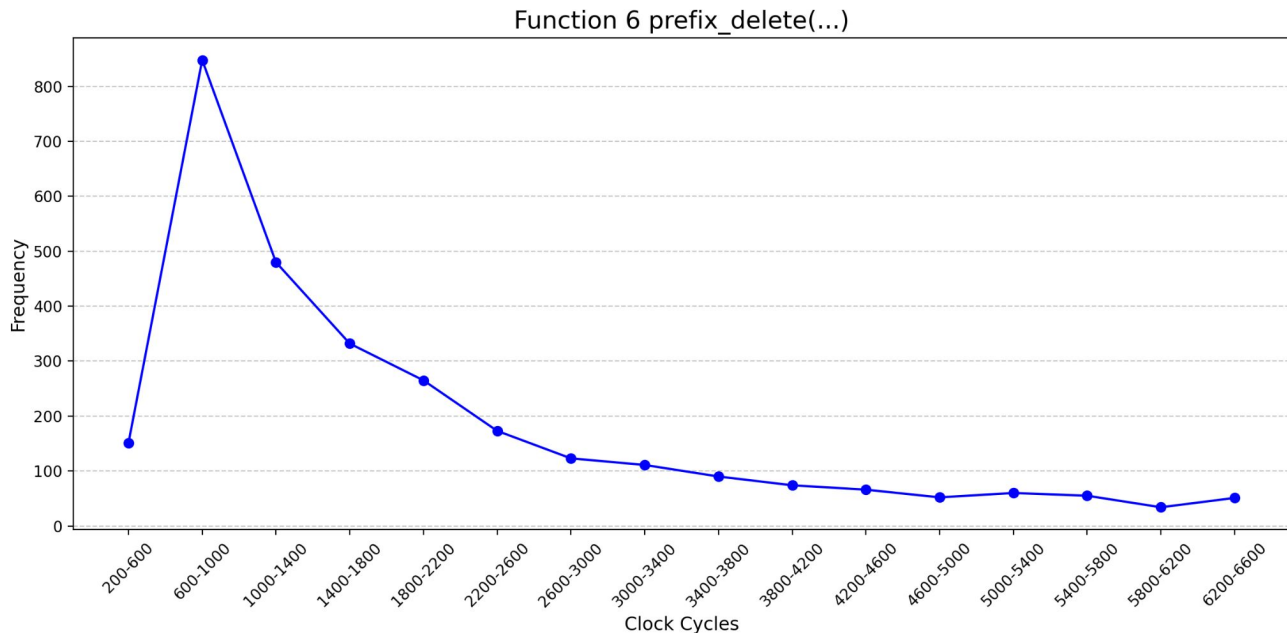
平均clock cycles = 2030 但是有離群值影響
大多時候程式的clock cycles都在800-1800間



```
mean      2030.168126
std       15160.598902
min        798.000000
25%       1197.000000
50%       1463.000000
75%       1995.000000
max      837368.000000
Name: clock cycles, dtype: float64
```

Function 6 prefix_delete(...)

平均clock cycles = 3240 但是有離群值影響
大多時候程式的clock cycles都在1000-4000間

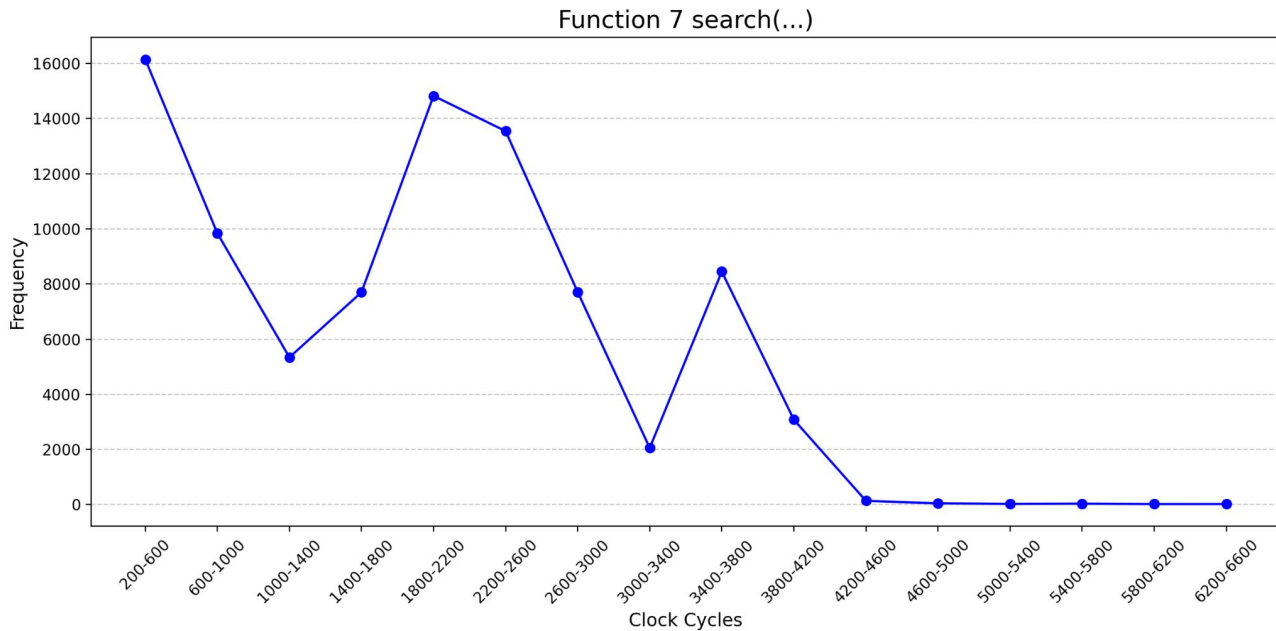


```
mean      3240.555523
std       13513.720422
min        21.000000
25%       931.000000
50%      1729.000000
75%      3857.000000
max     638799.000000
Name: clock cycles, dtype: float64
```

Function 7 search(...)

平均clock cycles = 1918 但是有離群值影響。

這個function的clock cycles比較分散 不過50%都在800-2500間。



```
mean      1918.736395
std       3092.194792
min       266.000000
25%       798.000000
50%      1995.000000
75%      2527.000000
max      683753.000000
Name: clock cycles, dtype: float64
```